CSE 402 — Online-2 Theory Revision

Linear systems (Gaussian / Gauss–Jordan) • LU Decomposition • Power Method for Eigenpairs

All formulas in math form • method-vs-method reasoning • worked simulation problems at the end

1. Gaussian Elimination & Gauss–Jordan

The idea

Solve $Ax = b$ by row operations. Reduce the augmented matrix $[A \mid b]$ to triangular form, then solve.

Naive Gaussian elimination

Eliminate below each pivot, then back-substitute.

$$m_{ik} = \frac{a_{ik}}{a_{kk}} \ (i > k), \qquad R_i \leftarrow R_i - m_{ik} R_k$$

Back-substitution (bottom-up):

$$x_i = \frac{1}{a_{ii}} \Big(b_i - \sum_{j=i+1}^{n} a_{ij} x_j \Big)$$

Why / vs previous: Simplest direct method. **Weakness:** breaks if a pivot $a_{kk} = 0$, and loses accuracy if a pivot is merely small (the multiplier m_{ik} blows up and amplifies rounding error).

Gaussian with partial pivoting

Before clearing column k, swap up the row with the largest-magnitude pivot candidate:

$$p = \arg\max_{i \geq k} |a_{ik}|, \qquad R_k \leftrightarrow R_p \ (\text{and } b_k \leftrightarrow b_p)$$

This forces $|m_{ik}| \leq 1$. **Why / vs previous:** **Better than naive:** same cost, but numerically *stable* — bounded multipliers keep rounding error from growing. This is the default you should always use. Downside: the row searches/swaps add bookkeeping, and it still fails only if a whole column is zero (then A is singular).

Gauss–Jordan

Normalize each pivot to 1 and eliminate *both above and below*, giving the identity on the left; the answer is read straight off:

$$R_k \leftarrow \frac{R_k}{a_{kk}}, \qquad R_i \leftarrow R_i - a_{ik} R_k \quad (\forall\, i \neq k)$$

Why / vs previous: **vs Gaussian:** no back-substitution needed, and it produces A^{-1} directly (augment with I instead of b). **Worse for a single solve:** $\sim n^3$ operations vs $\sim \frac{2}{3}n^3$ for Gaussian — about 50% more work. Use it when you actually want the inverse or reduced form, not just x.

Cost summary

$$\text{Gaussian: } \tfrac{2}{3}n^3 + n^2, \qquad \text{Gauss–Jordan: } \sim n^3$$

Classifying $Ax = b$

After elimination inspect the rows:

- no zero row ⇒ **unique** ($\operatorname{rank} A = n$)
- row $0 = c,\ c \neq 0$ ⇒ **no solution**
- row $0 = 0$ ⇒ **infinite** (free variables)

Rouché–Capelli: compare $\operatorname{rank}(A)$, $\operatorname{rank}([A|b])$ and n.

Edge cases / traps:

- Zero pivot but nonzero entry below → must pivot; not an error.
- Small (not zero) pivot → huge m_{ik} → catastrophic cancellation. Always pivot.
- In floating point, dividing by a 0 pivot gives $m = \infty$, then $\infty \cdot 0 = \text{NaN}$ silently.
- Every row swap must also be applied to b.

2. LU Decomposition

The idea

Factor once, reuse for many right-hand sides: $A = LU$ with L lower, U upper triangular. **Doolittle**: $\ell_{ii} = 1$. **Crout**: $u_{ii} = 1$.

$$\ell_{ik} = \frac{u_{ik}}{u_{kk}}, \qquad U_i \leftarrow U_i - \ell_{ik} U_k$$

Here U is exactly what Gaussian elimination produces; L just *stores the multipliers* that Gaussian elimination would have discarded.

Why / vs previous: **vs Gaussian elimination:** identical work for one solve, but the expensive $O(n^3)$ elimination is done once and *does not depend on* b. For k right-hand sides:

$$\underbrace{O(n^3 + kn^2)}_{\text{LU}} \ll \underbrace{O(kn^3)}_{\text{re-run Gauss}}$$

So LU wins big whenever you solve with the same A many times (also gives cheap determinant & inverse).

Solving with L, U

Let $z = Ux$. Two triangular solves:

$$Lz = b \ (\text{forward}), \qquad Ux = z \ (\text{backward})$$

$$z_i = \frac{1}{\ell_{ii}} \Big(b_i - \sum_{j<i} \ell_{ij} z_j \Big), \quad x_i = \frac{1}{u_{ii}} \Big(z_i - \sum_{j>i} u_{ij} x_j \Big)$$

Each solve is $O(n^2)$.

Existence (no pivoting)

$A = LU$ exists $\iff$ every leading principal minor is nonzero (first $n-1$):

$$\det(A_{1:k,\,1:k}) \neq 0, \quad k = 1, \ldots, n-1$$

$\det A \neq 0$ alone is *not* enough — e.g. $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ is invertible but has a zero 1×1 minor.

LU with partial pivoting: $PA = LU$

P = permutation recording swaps. Exists for *every* invertible A. **Why / vs previous: Better than plain LU:** fixes the zero-leading-minor failure and gives numerical stability ($|\ell_{ij}| \le 1$), just like partial pivoting did for Gaussian elimination. **Cost:** you must now solve $Lz = Pb$ (permute b first), not $Lz = b$.

Determinant & inverse

$$\det A = (-1)^{\#\text{swaps}} \prod_i u_{ii}$$

A zero on U's diagonal $\Rightarrow \det A = 0$ (singular). Inverse: solve $Ax = e_i$ for each unit column e_i; assemble the x's as columns of A^{-1}.

Cholesky (special case)

If A symmetric positive-definite ($A = A^T,\ x^T Ax > 0$):

$$A = LL^T$$

Why / vs previous: Better than LU for SPD: no pivoting needed (pivots stay positive) and about *half* the cost ($\sim \frac{1}{3}n^3$) and half the storage. Use whenever A is SPD (e.g. A^TA in least squares, covariance matrices).

Condition number

$$\kappa(A) = \|A\|\,\|A^{-1}\|, \qquad \frac{\|\delta x\|}{\|x\|} \lesssim \kappa(A)\,\frac{\|\delta b\|}{\|b\|}$$

A small residual $\|Ax - b\|$ does *not* guarantee small error when $\kappa(A)$ is large (ill-conditioned).

Edge cases / traps:

- Solve $Lz = Pb$, not $Lz = b$, once pivoting is used.
- A mid-elimination row swap must also swap the already-stored multipliers in L's earlier columns.
- Never form A^{-1} just to solve $Ax = b$ — it costs $\sim n\times$ more and worsens rounding.
- Zero on U diagonal = singular; back-substitution divides by it.

3. Power Method (Eigenpairs)

The idea

Find eigenvalues *without* the characteristic polynomial, by repeated multiplication. Any x_0 (with a component along v_1) gets stretched most along the dominant eigenvector.

Power iteration — dominant (λ_1, v_1)

Needs a strictly dominant eigenvalue $|\lambda_1| > |\lambda_2| \ge \cdots$

$$y = Ax, \qquad \lambda \approx y_{[\arg\max |y|\,]}, \qquad x \leftarrow \frac{y}{\lambda}$$

Convergence is linear with rate $|\lambda_2/\lambda_1|$. A sharper eigenvalue estimate is the **Rayleigh quotient**:

$$\lambda \approx \frac{x^T Ax}{x^T x}$$

Why / vs previous: Direct, matrix-free, cheap ($O(n^2)$ per step). **Limitation:** gives only the *largest* eigenvalue, and slowly if λ_2 is close to λ_1.

Inverse power — smallest $|\lambda|$

Eigenvalues of A^{-1} are $1/\lambda$, so power-iterate on A^{-1}:

$$\lambda_{\min}(A) = \frac{1}{\lambda_{\max}(A^{-1})}$$

In practice *solve* $Ay = x$ each step (via LU) rather than forming A^{-1}. **Why / vs previous: vs plain power:** targets the smallest eigenvalue, which plain power can never reach. Slightly costlier (a linear solve per step) but the LU factor is reused, so still cheap.

Shifted inverse power — any eigenvalue near μ

$$\text{iterate } (A - \mu I)^{-1} \ \Rightarrow\ \text{converges to } \lambda \text{ closest to } \mu$$

Rate $\propto \left|\frac{\lambda_1 - \mu}{\lambda_2 - \mu}\right|$. **Why / vs previous: Best of the three for a targeted eigenvalue:** a good shift μ makes convergence extremely fast, and it can reach interior eigenvalues neither of the others can.

Deflation — all eigenpairs (symmetric A)

After a converged pair, normalize $\hat{v} = v_1/\|v_1\|_2$ and remove it:

$$A' = A - \lambda_1\,\hat{v}\,\hat{v}^T$$

Then power-iterate A' for the next pair. Reconstruct $A = V\Lambda V^{-1}$. **Why / vs previous:** Turns a dominant-only method into a full spectrum method. **Downside:** rounding error accumulates, so later eigenpairs are least accurate.

Edge cases / traps:

- Use the largest-**magnitude** entry (keep its sign), not `max` — a negative dominant eigenvalue breaks a plain max.
- v and $-v$ are both valid; align signs before comparing.
- Fails / stalls if $|\lambda_1| = |\lambda_2|$ (e.g. a $\pm\lambda$ pair or complex conjugates).
- x_0 must not be orthogonal to v_1; normalize each step to avoid overflow.
- Converge on the *vector*, not just the value (the value settles first).
- Deflation needs a *unit* $\hat{v}$ ($\hat{v}^T\hat{v} = 1$).

Simulation / Worked Problems (with full solutions)

Q1 (Gaussian elimination with partial pivoting). Solve

$$\begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ -11 \\ -3 \end{pmatrix}.$$

Step 1 — column 1. Candidates $|2|, |-3|, |-2|$: largest is -3 (row 2). Swap $R_1 \leftrightarrow R_2$:

$$\left[\begin{array}{ccc|c} -3 & -1 & 2 & -11 \\ 2 & 1 & -1 & 8 \\ -2 & 1 & 2 & -3 \end{array}\right]$$

Multipliers $m_{21} = \frac{2}{-3} = -\frac{2}{3}$, $m_{31} = \frac{-2}{-3} = \frac{2}{3}$. Eliminate:

$$\left[\begin{array}{ccc|c} -3 & -1 & 2 & -11 \\ 0 & \frac{1}{3} & \frac{1}{3} & \frac{2}{3} \\ 0 & \frac{5}{3} & \frac{2}{3} & \frac{13}{3} \end{array}\right]$$

Step 2 — column 2. $|\frac{1}{3}| < |\frac{5}{3}|$, so swap $R_2 \leftrightarrow R_3$. Multiplier $m_{32} = \frac{1/3}{5/3} = \frac{1}{5}$:

$$\left[\begin{array}{ccc|c} -3 & -1 & 2 & -11 \\ 0 & \frac{5}{3} & \frac{2}{3} & \frac{13}{3} \\ 0 & 0 & \frac{1}{5} & -\frac{1}{5} \end{array}\right]$$

Back-substitution.

$$x_3 = \frac{-1/5}{1/5} = -1, \quad x_2 = \frac{\frac{13}{3} - \frac{2}{3}(-1)}{5/3} = \frac{5}{5/3} = 3, \quad x_1 = \frac{-11 - (-1)(3) - (2)(-1)}{-3} = 2.$$

$$\boxed{x = (2,\ 3,\ -1)}$$

Note every multiplier satisfies $|m| \le 1$ — the point of pivoting.

Q2 (LU decomposition — Doolittle, determinant & solve). For $A = \begin{pmatrix} 2 & -1 & -2 \\ -4 & 6 & 3 \\ -4 & -2 & 8 \end{pmatrix}$, find L, U, then $\det A$, then solve $Ax = (-1,\ 5,\ 2)^T$.

Factor. $\ell_{21} = \frac{-4}{2} = -2$, $\ell_{31} = \frac{-4}{2} = -2$; after clearing column 1 the $(3,2)$ multiplier is $\ell_{32} = \frac{-4}{4} = -1$. Result:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -2 & -1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & -1 & -2 \\ 0 & 4 & -1 \\ 0 & 0 & 3 \end{pmatrix}.$$

Determinant (no swaps): $\det A = \prod u_{ii} = 2 \cdot 4 \cdot 3 = \boxed{24}$.
Solve $Lz = b$ with $b = (-1, 5, 2)^T$ (forward):

$$z_1 = -1, \quad z_2 = 5 - (-2)(-1) = 3, \quad z_3 = 2 - (-2)(-1) - (-1)(3) = 3.$$

Solve $Ux = z$ (backward):

$$x_3 = \tfrac{3}{3} = 1, \quad x_2 = \tfrac{3-(-1)(1)}{4} = 1, \quad x_1 = \tfrac{-1-(-1)(1)-(-2)(1)}{2} = 1.$$

$$\boxed{x = (1,\ 1,\ 1)}$$

(Reusing L, U, any new b now costs only the two $O(n^2)$ solves.)

Q3 (Power iteration + Rayleigh quotient). For $A = \left(\begin{smallmatrix} 4 & 1 \\ 1 & 3 \end{smallmatrix}\right)$, $x_0 = (1, 0)^T$, do 3 iterations; compare with the exact dominant $\lambda = \frac{7+\sqrt{5}}{2} \approx 4.618$.

Scale by the largest-magnitude entry each step:

k	x_k	$\lambda^{(k)} = \max$-entry of Ax_k
0	(1, 0)	4
1	(1, 0.250)	4.25
2	(1, 0.412)	4.412
3	(1, 0.507)	4.507

Estimates climb toward 4.618 (rate $\lambda_2/\lambda_1 \approx 0.516$). **Rayleigh quotient** at $x_2 = (1, 0.412)$:

$$\frac{x^T A x}{x^T x} = \frac{5.332}{1.170} \approx 4.56,$$

already closer to 4.618 than the max-entry estimate 4.412 — Rayleigh converges faster.

Q4 (Inverse power — smallest eigenvalue). Same $A = \left(\begin{smallmatrix} 4 & 1 \\ 1 & 3 \end{smallmatrix}\right)$. Estimate the *smallest* eigenvalue.

$A^{-1} = \frac{1}{11}\left(\begin{smallmatrix} 3 & -1 \\ -1 & 4 \end{smallmatrix}\right)$ (since $\det A = 11$). Power-iterate A^{-1} from $x_0 = (1, 0)$:

$$(1, 0) \to (0.273, -0.091) \to \ldots \Rightarrow \lambda_{\max}(A^{-1}) \approx 0.420.$$

Then

$$\lambda_{\min}(A) = \frac{1}{0.420} \approx 2.38 = \frac{7 - \sqrt{5}}{2}. \checkmark$$

Matches the exact smaller root. (In code you would solve $Ay = x$ via LU instead of forming A^{-1}.)

Q5 (Deflation — second eigenpair). Using the converged dominant pair of $A = \left(\begin{smallmatrix} 4 & 1 \\ 1 & 3 \end{smallmatrix}\right)$, find λ_2 by one deflation step.

Dominant $\lambda_1 = 4.618$, direction $v_1 \propto (1, 0.618)$; normalize $\hat{v} = (0.851, 0.526)$, so

$$\hat{v}\hat{v}^T = \begin{pmatrix} 0.724 & 0.447 \\ 0.447 & 0.276 \end{pmatrix}.$$

$$A' = A - \lambda_1 \hat{v}\hat{v}^T = \begin{pmatrix} 0.658 & -1.065 \\ -1.065 & 1.724 \end{pmatrix}.$$

Power-iterating A' gives its dominant eigenvalue ≈ 2.382. Check via trace (the removed direction now has eigenvalue 0):

$$\operatorname{tr} A' = 0.658 + 1.724 = 2.382 = \lambda_2. \checkmark$$

So $\{\lambda_1, \lambda_2\} = \{4.618, 2.382\}$, matching $\frac{7 \pm \sqrt{5}}{2}$.

Q6 (Ill-conditioning — why residual can lie). Solve $A = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1.0001 \end{smallmatrix}\right) x = \left(\begin{smallmatrix} 2 \\ 2.0001 \end{smallmatrix}\right)$, then change b_2 to 2.0002 and resolve.

First system: subtract R_1 from $R_2 \Rightarrow 0.0001\, x_2 = 0.0001 \Rightarrow x_2 = 1,\ x_1 = 1$, so $x = (1, 1)$. Perturbed: $0.0001\, x_2 = 0.0002 \Rightarrow x_2 = 2,\ x_1 = 0$, so $x = (0, 2)$. A change of 10^{-4} in b moved x by order 1:

$$\kappa(A) \approx \frac{\|\delta x\|/\|x\|}{\|\delta b\|/\|b\|} \sim 4 \times 10^4.$$

Lesson: a tiny residual $\|Ax - b\|$ does not imply an accurate x when $\kappa(A)$ is large.

One-line recall: *pivot for stability • leading minors for LU existence • permute b when $PA = LU$ • largest-magnitude scaling for power • never invert just to solve • check $\kappa(A)$, not just the residual.*